

Catalog Cover Images Not Loading — Root Cause Analysis

Revision: 1 **Last modified:** 2026-06-29T13:30:00Z **Reported by:** Operator (escaped the green HelixQA campaign — §11.4.138)
Authority: §11.4.102 systematic-debugging + §11.4.6 no-guessing + §11.4.123 rock-solid-proof + §11.4.145 impact-research

Symptom

Every catalog item on Android TV (and all clients) shows a placeholder gradient + “unknown” badge instead of a real cover/poster image. `cover_url` for every item is `/api/v1/cover/placeholder/<type>`.

Root cause (FACT — captured evidence, not guess)

The cover pipeline is **correct**; there is simply **no cover data to serve**. Proven via direct PostgreSQL queries (catalogizer_dev DB):

Table	Count	Meaning
media_items	27750	full scanned catalog
external_metadata	0	NO enrichment data exists for ANY item
cover_art	0	NO cached covers
directory_analyses	4049	only 4046 items are reachable by the enrichment query

GetCoverURL (cover_art_service.go:1446) is a passive lookup: checks `cover_art` then `external_metadata.cover_url`; both empty → returns placeholder. Placeholder-on-no-data is the *designed* fallback (cover_handler_test.go confirms), NOT a bug.

Why external_metadata is empty — the terminal cause

EnrichAllEntities (media_entity_handler.go:833) tries, per item:
(1) local cover file (cover.jpg/folder.jpg/poster.jpg in the item dir — none present on these SMB shares), (2) TMDB (TMDB_API_KEY — **not configured**), (3) LLM fallback.

Captured API log (/tmp/catalog-api.log) — every enrichment attempt:

```
WARN handlers/media_entity_handler.go:810 LLM fallback failed for
'Phone Booth': LLM API returned HTTP 402
WARN handlers/media_entity_handler.go:810 LLM fallback failed for
'The Guilty': LLM API returned HTTP 402
INFO handlers/media_entity_handler.go:991 EnrichAllEntities:
completed - 0 enriched (0 local, 0 TMDB) out of 10 queued
```

HTTP 402 = Payment Required. The LLM provider the API selected (DeepSeek, priority-1) is out of credit. Direct probes of every configured LLM provider:

Provider (priority order)	Chat probe	Status
DeepSeek (selected)	HTTP 402	out of credit
OpenRouter	HTTP 402	out of credit
Groq	HTTP 200	funded & usable
Gemini	HTTP 400	payload-format (may work)
Kimi	HTTP 401	bad/expired key

Two distinct defects

1. **CREDENTIAL (operator):** no movie-poster source is usable — TMDB key absent; the chosen LLM (DeepSeek) is out of credit. The *correct* poster source is TMDB (a real poster CDN); LLM-generated URLs are best-effort and may hallucinate/404.
2. **CODE (§11.4.145 impact-research catch):** LLMProvider factory (internal/media/providers/llm_provider.go:46) binds to the FIRST non-empty key (DeepSeek) for the process lifetime and has **no failover**. When DeepSeek 402s, it gives up — even though **Groq (HTTP 200) is funded and available**. A resilient design would fail over to the next candidate on 402/429/5xx.

3. **ARCHITECTURAL (secondary)**: the enrichment background query JOINS `directory_analyses`, so only 4046 of 27750 items are even reachable — 85% of the catalog cannot be enriched by the current endpoint regardless of credentials.

Fix direction (pending operator decision)

- **Option A (best quality)**: operator adds a `TMDB_API_KEY` to `.env` → real posters from the TMDB CDN. Plus code failover (Option B) as resilience.
- **Option B (zero operator action, code-only)**: implement LLM-provider failover so enrichment uses the funded Groq provider instead of the dead DeepSeek. Restores covers via best-effort LLM URLs (quality caveat — must verify URLs resolve, §11.4.123).
- **Option C (both)**: TMDB primary + LLM failover fallback + fix the `directory_analyses` reachability gap so all 27750 items can be enriched.

Evidence files

- `/tmp/catalog-api.log` — the HTTP 402 enrichment failures (captured).
- DB query output (above) — `external_metadata=0`, `cover_art=0`.
- LLM provider probe results (above).

Resolution (FIXED — captured end-to-end proof)

Operator supplied a working TMDB key + chose “fix reachability for all 27750 items”. Applied THREE fixes, all TDD RED→GREEN, go vet clean:

1. **TMDB key** stored in gitignored `.env` (leak-audited, verified untracked). Verified live: HTTP 200, real posters (Iron Man → `/78lPtww72e...jpg`).
2. **Reachability** (`media_entity_handler.go`): inner JOIN → LEFT JOIN on `directory_analyses` so all 27750 items reachable (was 4046). 7/7 tests pass.
3. **LLM failover** (`llm_provider.go`): DeepSeek 402 → fails over to funded Groq. 5 tests pass.
4. **Progress marker** (in progress): enrichment was reprocessing the same unmatchable leading items forever (no “tried”

sentinel) — being TDD-fixed so the queue advances to the real movies.

End-to-end captured proof (§11.4.123): - DB: external_metadata 0 → 15 (and climbing), all with real cover URLs. - API log: TMDB now queried successfully (no more HTTP 402); EnrichAllEntities: completed – 2 enriched (2 TMDB). - Cover bytes: “Phone Booth” entity cover_url → image.tmdb.org/.../r6lI...jpg; the image-proxy serves a **64,105-byte real image/jpeg** (was 1,398-byte placeholder SVG). - TMDB confirmed to match all visible movies (Iron Man, Captain America, Avengers) with real poster_paths.

FINAL RESULTS (covers populated at scale)

Five composable TDD fixes (all RED→GREEN, go vet clean, all pushed): 1. TMDB key (gitignored .env) + LEFT-JOIN reachability (all 27750 items) 2. LLM-provider failover (DeepSeek 402 → funded Groq) 3. Progress-marker sentinel (queue advances past unmatchable items) 4. Movie/TV type-priority + recency ordering (visible shelf covered first) 5. Title normalization (cleanTitleForSearch — strips “001 -” prefixes, [release]/(scene) tags that broke TMDB matching)

Captured end-to-end proof (§11.4.123) — FINAL clean numbers (distinct, post-dedup, post-reclassify): - external_metadata: 0 → **722/813 real movies (89%) + 167/177 TV shows (94%)** distinct TMDB covers (COUNT(DISTINCT media_item_id)). The remaining ~11% are genuinely obscure/foreign titles TMDB lacks, correctly marked enrichment_attempted. - DATA-INTEGRITY corrections applied this session (§11.4.6, anti-bluff): - An earlier draft cited “1153/1192 (97%)” — a non-distinct JOIN count read during active concurrent enrichment. The honest distinct figure is the 89%/94% above. - The “1192 movies” total was itself inflated: **379 of them were TV episodes** (S01E09, “Episode N”) that the scanner misclassified as movies. Fixed in code (conservative leading-anchored detector in aggregation_service.go, TDD) AND retroactively in the DB (379 rows movie→tv_episode), so the real-movie denominator is 813, not 1192. - The duplicate-row issue (Upsert read-then-write race) is now **structurally prevented**: migration v20 adds a self-healing dedup + UNIQUE INDEX on (media_item_id, provider), and Upsert is conflict-tolerant. 0 duplicate rows remain. - The exact marquee title the operator saw missing — “001 - Captain America - The First Avenger” (0 TMDB matches with its raw noisy title) — now serves a **real 101,336-byte image/jpeg** poster (image.tmdb.org/t/p/w500/vSNxAJ...jpg) via the cleaned query “Captain America The First Avenger”.

Eight composable fixes total (the five above + dedup-prevention unique index + episode classification + transaction context-lifetime determinism fix), all TDD RED→GREEN, go vet clean, pushed to all 6 remotes. Infrastructure also made resilient: restart: unless-stopped on the dev pod (postgres no longer silently dies) + POSTGRES_DB config fallback.

Honest boundary (§11.4.6)

This was NOT a cover-rendering bug — the fix is data-pipeline + credential, not UI code. The pipeline is PROVEN end-to-end at the DB + API + image-bytes layers. Device-visual confirmation requires an app re-login (force-stop clears the saved session; the persisted server URL is the unreachable LAN IP — use the §11.4.117 pixel-oracle with localhost:8080 + adb reverse tcp:8080 tcp:28080). The covers ARE in the database and serve as real JPEGs; they render on the app's next authenticated catalog fetch.